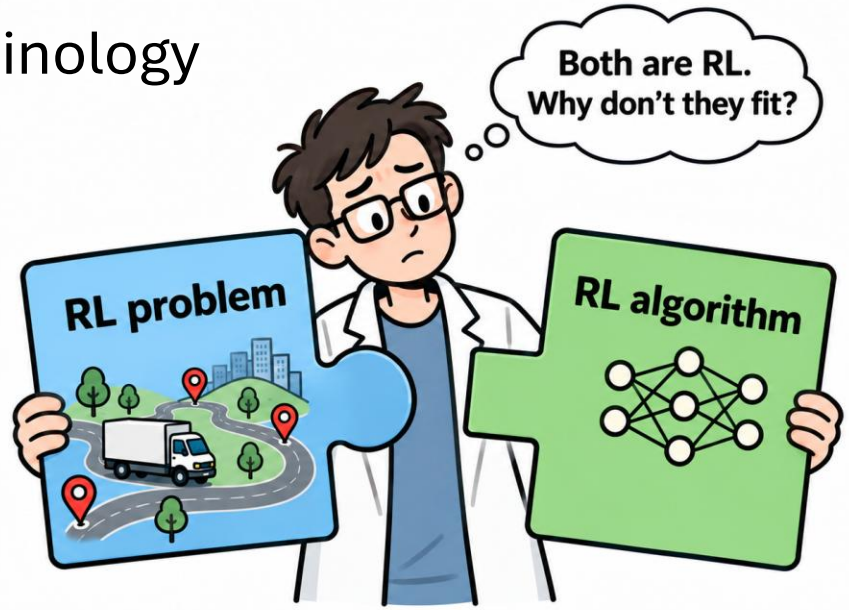
Practical RL:
How applications drive algorithm design

Ido Greenberg

2026

Agenda

- Yesterday: Tabular RL → Deep RL → PPO → cute GIFs
- What else? Real application → different setup → modified algo
- This talk: Notice setup aspects & terminology
  - Next talks: Solve some of them



Train framework: where does data come from?



Train framework: where does data come from?

	Behavior policy = Learned policy?
On-policy	Yes
Off-policy	No E.g. older / explorative



Train framework: where does data come from?

	Behavior policy = Learned policy?	Interactive?
On-policy	Yes	Yes
Off-policy	No E.g. older / explorative	
Offline	No	**No**



Examples: learn how to trade

- **Offline:** learn from blue data
- **On/off-policy:** learn from simulation based on blue data

Train framework: where does data come from?

	Behavior policy = Learned policy?	Interactive?	Train before inference
Online		Yes	**Train = inference** ⇒ exploration / exploitation tradeoff
On-policy	Yes	Yes	Yes
Off-policy	No E.g. older / explorative		
Offline	No	No	



Examples: learn how to trade

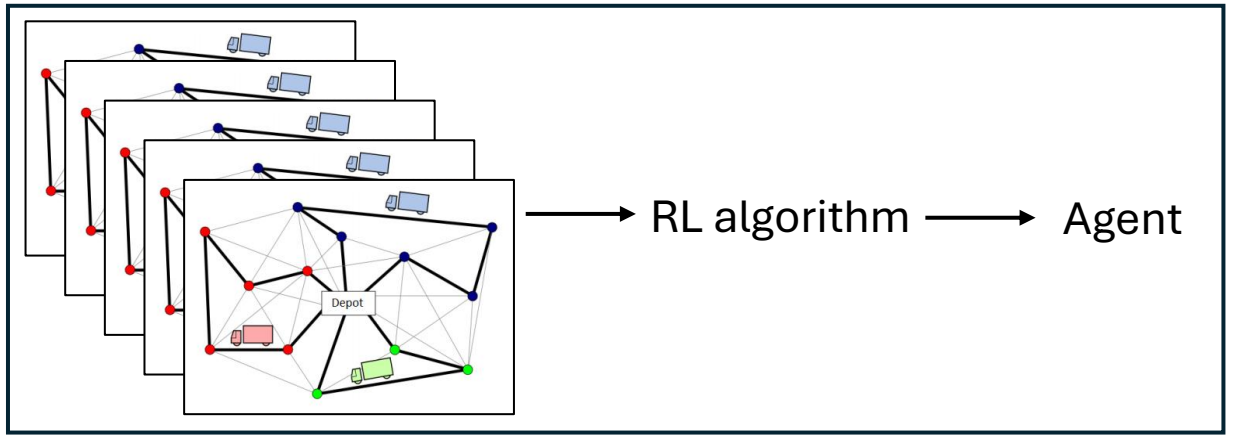
- **Offline:** learn from blue data
- **On/off-policy:** learn from simulation based on blue data
- **Online learning:** learn from red data as you trade

Training bottleneck: data vs. processing

- **Processing bottleneck:** time / compute
  - Common case: fast simulator
  - Example: TSP solver (Traveling Salesman)
    - Prioritize data simulation & scaling
    - Use fast optimizer
    - Parallelize
    - Strong GPU







Training bottleneck: data vs. processing

- **Processing bottleneck:** time / compute
- **Data bottleneck:** interaction with environment
  - Example: physical robots
    - Prioritize sample-efficiency, do more planning / optimization
    - **World models**



Inference bottleneck: time vs. complexity

- **Bottleneck: time**

- **Bottleneck: complexity**

Inference bottleneck: time vs. complexity



- **Bottleneck: time**
  - Example: real-time Switch configuration – micro-sec decisions
  - Minimize input & model
  - Simplify settings (e.g. 1-step multi-arm bandits)



- **Bottleneck: complexity**

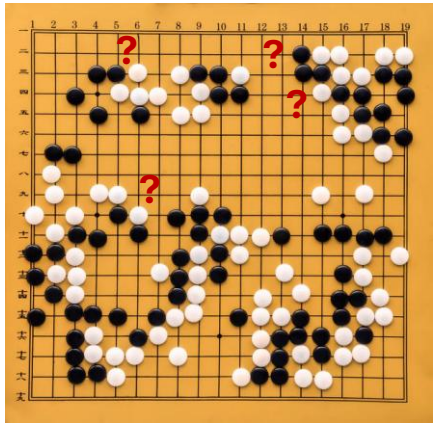
Inference bottleneck: time vs. complexity

- **Bottleneck: time**
  - Example: real-time Switch configuration – micro-sec decisions
  - Minimize input & model
  - Simplify settings (e.g. 1-step multi-arm bandits)



- **Bottleneck: complexity**
  - Example: board games
  - Leverage inference time for better decisions
    - E.g. simulate different actions and choose by predicted outcome
  - Training ⇔ inference protocol





Exploration

- Local optimum in RL
  - Any *slight* change only hurts $(\nabla_\theta E[R] = 0)$



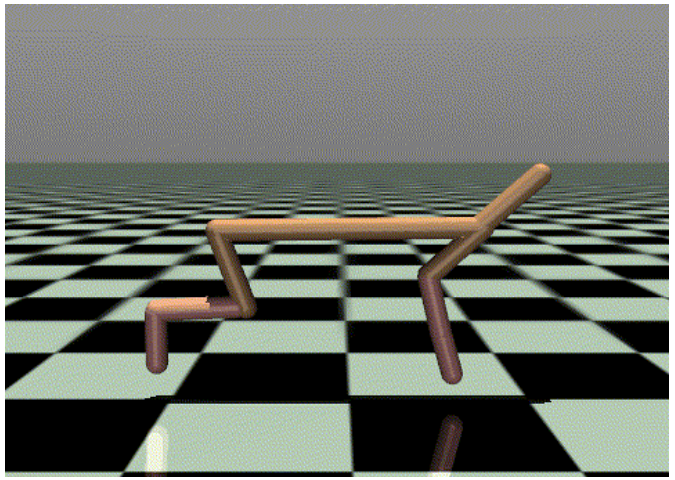
Goal: get close to target

Exploration

- Local optimum in RL
  - Any *slight* change only hurts



Goal: get close to target



Goal: run fast

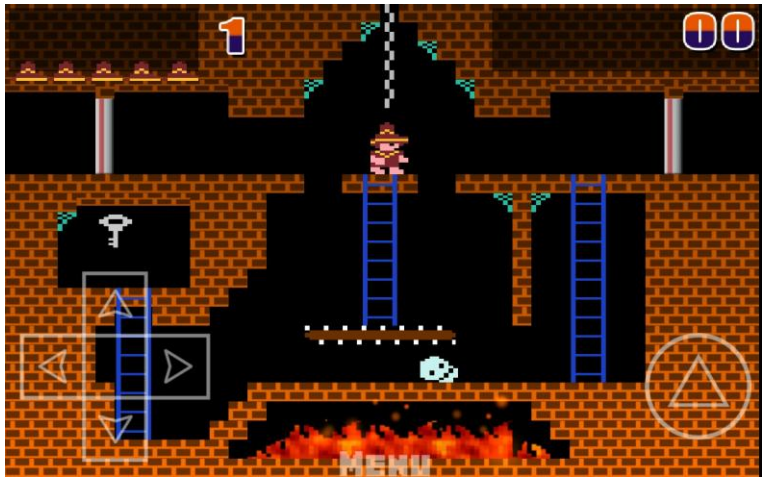
Exploration

- Local optimum in RL
  - Any *slight* change only hurts

- How to force looking for new behaviors?
  - Add randomness (*ϵ-greedy,* entropy bonus)

Exploration

- Local optimum in RL
  - Any *slight* change only hurts

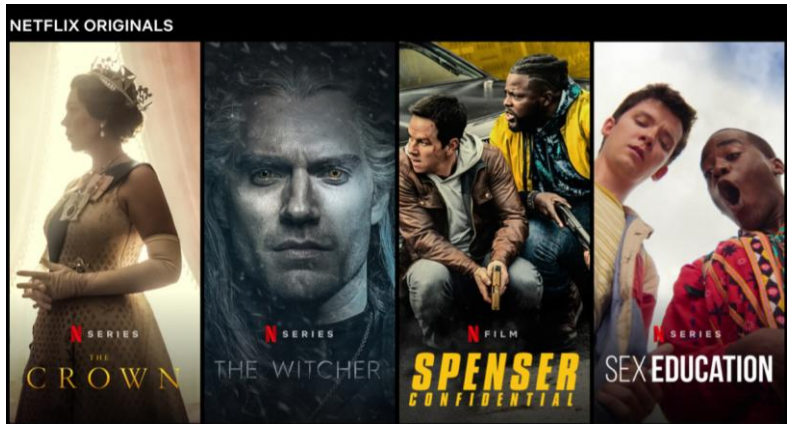


Montezuma's Revenge (Atari)

- How to force looking for new behaviors?
  - Add randomness (*ϵ-greedy,* entropy bonus)
  - Bonus for new observations (*RND*)

Exploration

- Local optimum in RL
  - Any *slight* change only hurts



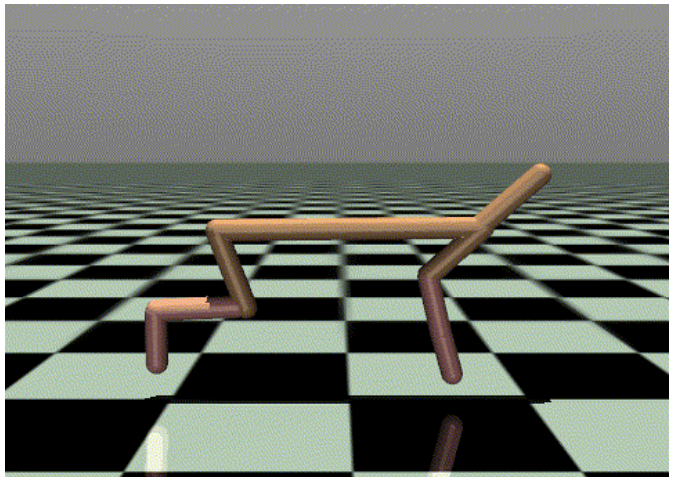
30/100 clicks | 17/50 clicks | 10/40 clicks | 0/2 clicks

Little data, high uncertainty

- How to force looking for new behaviors?
  - Add randomness (*ϵ-greedy,* entropy bonus)
  - Bonus for new observations (*RND*)
  - Optimism under uncertainty (R_{max}, *UCB*)

Exploration

- Local optimum in RL
  - Any *slight* change only hurts



Goal: run fast

- How to force looking for new behaviors?
  - Add randomness (*ϵ-greedy,* entropy bonus)
  - Bonus for new observations (*RND*)
  - Optimism under uncertainty (*R_{max}*, *UCB*)
  - Use domain knowledge (*reward shaping*)

Exploration

- Local optimum in RL
  - Any *slight* change only hurts



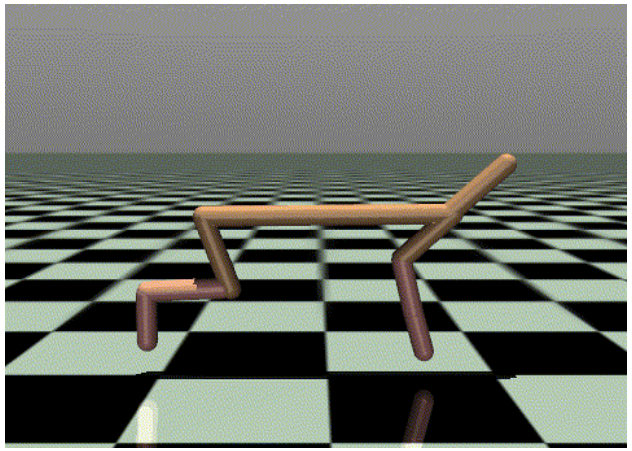
- How to force looking for new behaviors?
  - Add randomness (*ϵ-greedy,* entropy bonus)
  - Bonus for new observations (*RND*)
  - Optimism under uncertainty (R_{max}, *UCB*)
  - Use domain knowledge (*reward shaping*)
  - Start from a good initial policy (start with supervised / curriculum)

Reward design

- Wrong reward ⇒ ***Reward Hacking***
  - Reward = user score ⇒ learning to tell (wrong) fun facts
  - Reward = winning a game ⇒ learning to hack the game code

Reward design

- Wrong reward ⇒ ***Reward Hacking***
- Correct reward ⇒ sometimes hard to learn
  - Correct math proof ⇒ usually $r = 0$ (***sparse reward***)
  - Run fast ⇒ run on head (***local optimum***)



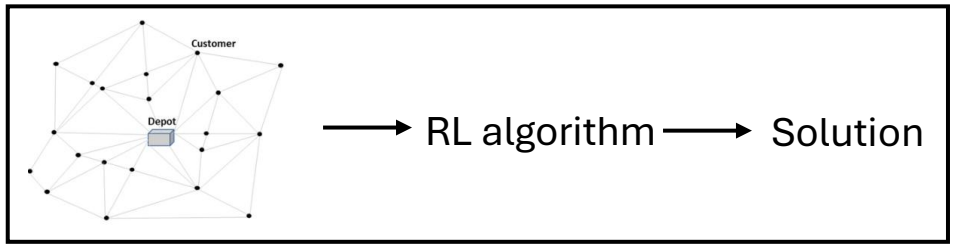
Reward design

- Wrong reward ⇒ ***Reward Hacking***
- Correct reward ⇒ sometimes hard to learn
  - ***Reward Shaping***: make the reward learnable

Scope & Generalization

- Optimize per instance (*Search*)
  - Train via RL on every new instance

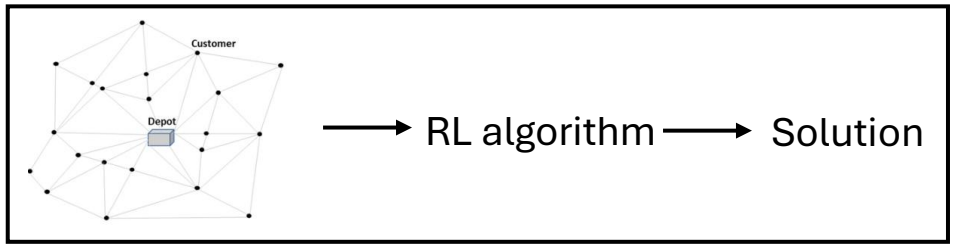
Example: routing problems



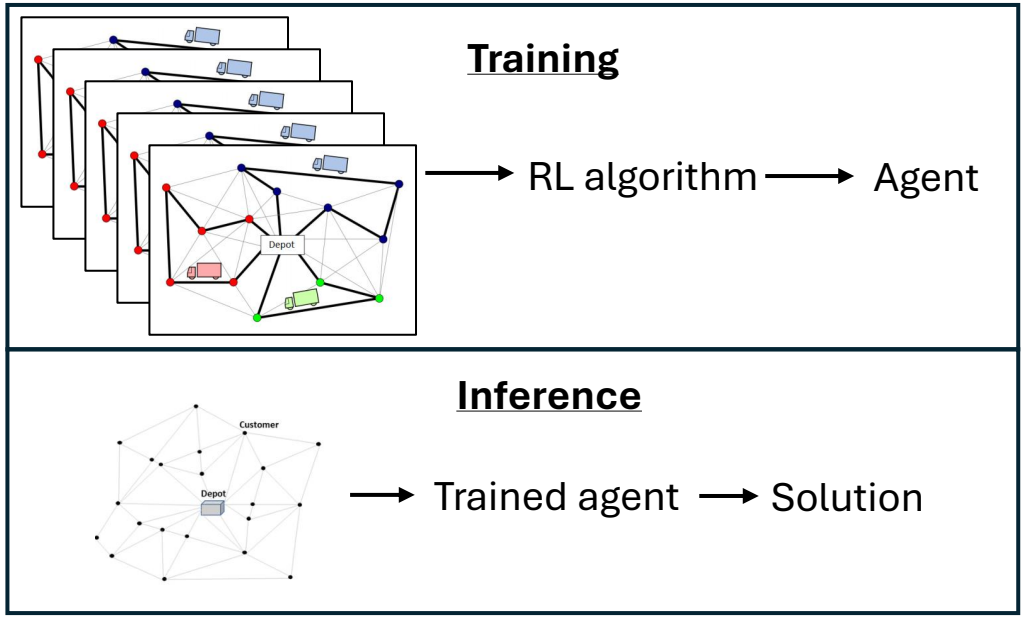
Scope & Generalization

- Optimize per instance (*Search*)
  - Train via RL on every new instance

Example: routing problems



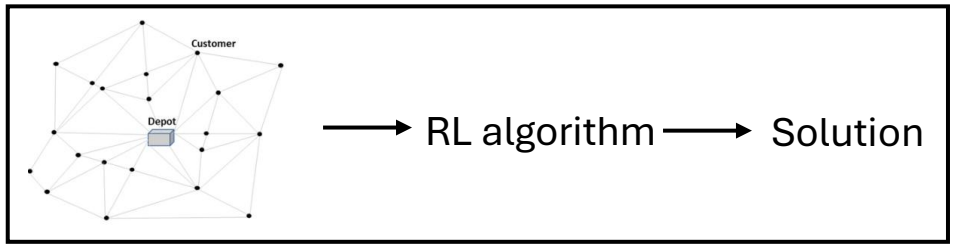
- Generalizing agent
  - Train on many instances
  - Inference on new instances
  - Fast but sensitive to data



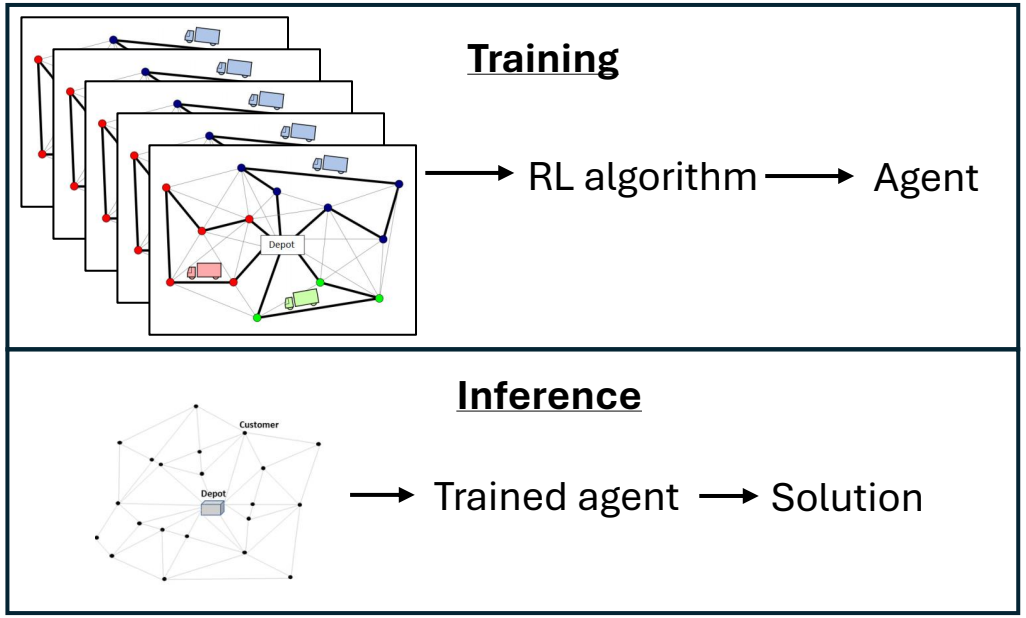
Scope & Generalization

- Optimize per instance (*Search*)
  - Train via RL on every new instance

Example: routing problems

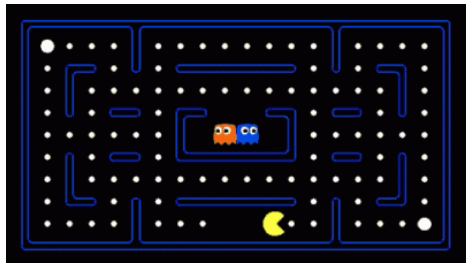


- Generalizing agent
  - Train on many instances
  - Inference on new instances
    - **Option I:** zero-shot
    - **Option II:** fine-tune (***meta-RL***)



Multi Agent RL: When you're not the only agent

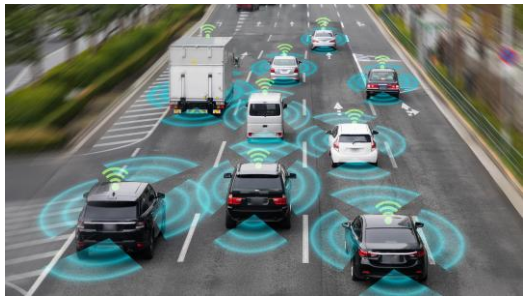
- **Competitive**
  - Competitors ≈ part of the env…
    → unless they adapt to you (adversarial non-stationarity)
  - Goal: ~~Optimization~~ **Equilibrium**





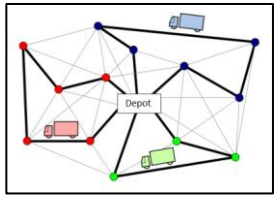


- **Cooperative**
  - Cooperators ≈ one single agent…
    → unless info sharing is limited
  - **Decentralized policy**: **Coordination**



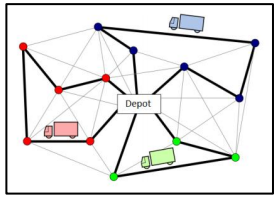
Other properties

- Episodes fixed len / varying len / lifelong



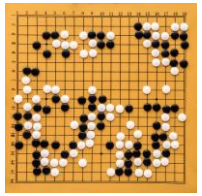


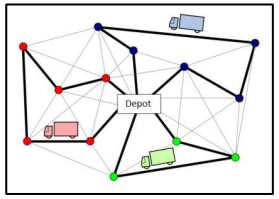

- Time regular steps / irregular / continuous






- Action space small / large / continuous / varying



Summary: application drives the algorithm

• Training framework	online / on-policy / off-policy / offline
• Training bottleneck	compute / data
• Inference bottleneck	time / complexity
• Exploration	avoid poor local optima
• Reward	correct / learnable
• Generalization	optimize once / per instance
• Multi agent	equilibrium / coordination